END-TO-END MLOps

Heart Disease Classification Pipeline

Machine Learning Operations (MLOps) — AIMLCZG523
Assignment 01 | Total Marks: 50

Project scope

Data acquisition and EDA; feature engineering; model development and validation; MLflow experiment tracking; reproducible packaging; CI/CD; Docker API; Kubernetes deployment; monitoring and documentation.

Dataset: UCI Machine Learning Repository — Heart Disease, processed Cleveland subset (303 records, 13 predictors). DOI: 10.24432/C52P4X.

Health and safety note: This project is an educational demonstration only. It is not intended for medical diagnosis or clinical decision-making.

Prepared as a reproducible project template. Runtime metrics, CI screenshots, deployment evidence, and public repository details must be populated from a genuine execution.

1. Executive Summary and Objectives

This project demonstrates an end-to-end MLOps lifecycle for a binary classification task using the processed Cleveland Heart Disease dataset. The target is derived from the original UCI `num` field: 0 represents no diagnosed disease in the dataset, while values 1–4 are mapped to presence. The system is designed to make preprocessing, evaluation, experiment tracking, packaging, serving, and deployment repeatable.

The implementation compares Logistic Regression and Random Forest pipelines. A stratified train/test split is used, and model candidates are evaluated with five-fold stratified cross-validation and grid search using ROC-AUC. Accuracy, precision, recall, F1-score, and ROC-AUC are recorded. The selected preprocessing transformer and classifier are serialized as a single pipeline.

The delivery includes source modules, tests, GitHub Actions, Docker and Compose files, Kubernetes manifests, Prometheus metrics, setup documentation, and this report. The project deliberately avoids claiming metrics before the dataset has been downloaded and the scripts executed in the target environment.

Objectives

• Reproduce data loading and cleaning from a documented source.
• Inspect distributions, class balance, missingness, and correlations.
• Compare two model families with leakage-safe validation.
• Log experiment parameters, metrics, and artifacts in MLflow.
• Package a validated prediction API and deploy it locally or in a cloud cluster.
• Add automated quality checks and observable runtime behavior.

Scope and assumptions

The implementation uses Python 3.11 and a local MLflow file store. Internet access is needed for first-time UCI data acquisition. Kubernetes deployment instructions target Minikube as a low-cost local option; a public cloud endpoint requires a separately provisioned cluster, registry, networking, and credentials.

2. Data Acquisition and Exploratory Analysis

The official UCI Heart Disease repository documents four databases and a commonly used processed Cleveland subset. This project downloads `processed.cleveland.data`, assigns the 13 predictor names and original `num` target, and treats question marks as missing values. The target is binarized as described above. Duplicate rows are removed, while feature missingness is retained for imputation inside the model pipeline.

Data acquisition command

```
python -m src.data_processing
python -m src.validate_data
python -m src.eda
```

EDA outputs

The EDA module creates a multi-panel feature distribution figure, a class-balance chart, and a correlation heatmap. It also writes `eda_summary.json` with row count, feature count, class counts, missing values by column, and the target definition. These are generated from the actual downloaded dataset at runtime.

Interpretation plan

• Check numeric ranges and unusual values before modelling.
• Compare the two target class counts to understand imbalance.
• Review pairwise correlations as descriptive associations, not causal relationships.
• Verify missing-value patterns and ensure imputation is learned only from training folds.
• Avoid claims of population representativeness: the Cleveland subset is small and historical.

Evidence limitation

The source PDF specifies EDA visualizations but does not provide computed summary statistics. The figures and exact values should therefore be generated by running the included scripts; no unexecuted counts or plot interpretations are invented in this report.

3. Feature Engineering and Model Development

The pipeline keeps preprocessing attached to each estimator. Numeric columns use median imputation followed by standard scaling. Categorical/integer-coded clinical attributes use most-frequent imputation followed by one-hot encoding with unknown categories ignored. This treatment allows the model to handle missing values and avoids imposing an artificial ordinal distance between category codes.

Candidate models

Model	Rationale	Tuning
Logistic Regression	Interpretable linear baseline; class weights mitigate imbalance.	$C \in \{0.1, 1, 10\}$; max_iter=3000
Random Forest	Captures nonlinear relationships and interactions.	Trees $\in \{150, 300\}$; depth $\in$ {None, 5, 10}; leaf $\in \{1, 3\}$

Evaluation design

A stratified 80/20 split maintains approximate class proportions. Five-fold StratifiedKFold validation is applied to the training partition. GridSearchCV selects each candidate's hyperparameters by mean ROC-AUC. The evaluation script records accuracy, precision, recall, F1-score, ROC-AUC, and CV ROC-AUC mean and standard deviation, then generates confusion-matrix and ROC plots.

Methodological caveat

For this compact assignment workflow, the held-out partition is used to compare the two tuned candidates. For a strictly unbiased final performance estimate, use training data for fitting, validation data for selection, and a separate untouched test set for one final evaluation. Report this distinction when presenting results.

4. Experiment Tracking and Reproducibility

MLflow is configured with the experiment name `heart-disease-classification`. Each candidate run records model family, split size, random seed, cross-validation fold count, CV metric means and standard deviations, selected grid-search parameters, and held-out metrics. The fitted sklearn pipeline is logged as a model artifact, while classification reports are saved as JSON artifacts.

Launch the tracking UI

```
mlflow ui --backend-store-uri ./mlruns --host 127.0.0.1 --port 5000
```

Open http://127.0.0.1:5000 to compare runs. The local `mlruns/` directory is ignored by Git because it can grow over time; export or upload it as a CI artifact when evidence is required.

Reusable packaging

The final preprocessing-plus-classifier pipeline is serialized with joblib to `models/heart_disease_pipeline.joblib`. Keeping transformations and estimator together prevents inference-time preprocessing from drifting away from training. The model card records the selected model, target definition, feature order, evaluation metrics, best parameters, and limitations. Requirements are bounded by major versions; a fully frozen lock file should be generated after a successful tested installation for long-term repeatability.

Reproducibility controls

• Fixed random seed for splitting, cross-validation, and estimators.
• Stratified folds and stratified holdout.
• Pipeline-based preprocessing to reduce leakage.
• Explicit feature list and target mapping.
• Machine-readable metric/model-card artifacts.
• Script-based execution to support CI and clean environments.

5. Automated Testing and CI/CD

The pytest suite checks that preprocessing can impute missing numeric values and transform categorical variables, and that the FastAPI service exposes its root and health endpoints. Dataset validation checks required columns, non-empty input, and a binary target. Ruff provides a linting gate.

Workflow sequence

```
Checkout source
  -> Set up Python 3.11
  -> Install requirements
  -> Ruff lint
  -> Download and validate dataset
  -> Pytest
  -> Generate EDA outputs
  -> Train and log models
  -> Build Docker image
  -> Upload run artifacts
```

The GitHub Actions workflow runs for pushes to main/master, pull requests, and manual dispatch. Errors in the required steps fail the workflow; artifacts are uploaded even when later steps fail when available. The CI job needs internet access to retrieve the dataset and Python packages.

How to verify

Push the repository to GitHub, open the Actions tab, select the latest workflow run, and confirm each step's status. Download the `mlops-run-artifacts` bundle and capture a screenshot of the successful run. If a job fails, inspect logs and fix the underlying issue rather than editing the screenshot or suppressing the failure.

Evidence not embedded

No GitHub repository or workflow run was supplied at report-generation time. A real run screenshot cannot be included until the project is pushed and executed in a GitHub account.

6. Model Serving API and Containerization

FastAPI provides a JSON-based `/predict` endpoint with typed input validation. The 13 expected features are explicit in the request schema. The response includes a binary prediction, human-readable dataset label, predicted-class confidence, estimated model probability for the positive class, and a non-clinical-use disclaimer. `/health` checks whether the model file exists; `/metrics` exposes Prometheus instrumentation.

Run locally

```
python -m src.eda
python -m src.train
uvicorn src.api:app --host 127.0.0.1 --port 8000
```

Visit `/docs` for an interactive OpenAPI interface. Use the README's sample JSON to smoke-test `/predict`. The sample is illustrative and not a medical recommendation.

Docker packaging

```
docker build -t heart-disease-api:latest .
docker run --rm -p 8000:8000 heart-disease-api:latest
curl http://localhost:8000/health
```

The Docker image uses Python 3.11 slim, installs the requirements, copies application source and model artifacts, exposes port 8000, and starts Uvicorn. A health check probes `/health`. The image build should happen after training so the serialized model is present.

Operational safeguards

The service logs outcomes and latency but does not log raw feature payloads. In production, add authentication, TLS, rate limiting, secret management, controlled access to metrics, and a retention policy. Input validation limits reduce malformed requests but do not establish medical validity.

7. Production Deployment

The repository includes a Kubernetes Deployment with two replicas, resource requests/limits, readiness and liveness probes, and a LoadBalancer Service. This supports a local Minikube deployment or a cloud Kubernetes cluster after the container image is made available to the cluster.

Minikube workflow

```
minikube start
docker build -t heart-disease-api:latest .
minikube image load heart-disease-api:latest
kubectl apply -f deployment/deployment.yaml
kubectl rollout status deployment/heart-disease-api
kubectl get pods,svc
minikube service heart-disease-api --url
```

For local Minikube, a LoadBalancer external IP may remain pending. The `minikube service` command or a `kubectl port-forward` tunnel can be used for local verification. For cloud deployment, push the image to a registry, update the manifest image reference, configure the cluster and ingress/network policies, and use managed secrets.

Verification plan

1. Confirm rollout completed and pods are Ready.
2. Open `/health` and confirm the model is available.
3. Open `/docs` and submit a sample prediction request.
4. Inspect service logs and `/metrics` output.
5. Record pod/service output and browser/API evidence for the submission.

Deployment status

The manifest and instructions are included, but this environment did not execute Minikube or create a cloud service. No public URL or deployment screenshot is claimed. Complete those steps in an environment with Docker/Kubernetes installed.

8. Monitoring, Logging, Risks, and Limitations

The API exposes Prometheus metrics for total prediction requests by outcome and request duration. Structured logs are emitted to standard output with event type, outcome, client address, prediction class, and duration. Docker and Kubernetes can collect these logs using standard runtime logging mechanisms. The included Prometheus configuration provides a starting scrape target.

Useful operational checks

• API availability: health endpoint and container/Kubernetes probes.
• Request volume and failures: `prediction_requests_total` grouped by status.
• Latency: `prediction_request_duration_seconds` histogram.
• Exceptions: inspect application logs for stack traces and model-load errors.
• Data/model drift: not implemented as an automated detector; add reference distributions, approved monitoring data, and alert thresholds before production use.

Limitations and responsible use

The dataset is small, historical, and not representative of all populations or current clinical practice. Metrics on this dataset cannot establish clinical validity, fairness, safety, or generalization. Probability output is not necessarily calibrated and should not be described as an individual's medical risk. Do not submit identifiable patient data to this demo. Real deployment would require privacy/security controls, governance, model validation, and domain-expert review.

Security notes

Do not expose the API or Prometheus endpoint publicly without appropriate controls. Avoid retaining patient attributes in logs. Use TLS, authentication, authorization, restricted network policies, secret management, vulnerability scanning, and a patching process. The current assignment implementation is a learning scaffold, not a production-certified healthcare system.

9. Conclusion, Evidence Checklist, and References

This project provides the main artifacts for an end-to-end MLOps workflow: data ingestion and EDA, two model pipelines, cross-validation and tuning, MLflow tracking, model serialization, tests, CI workflow, FastAPI serving, Docker packaging, Kubernetes manifests, and monitoring hooks. The source is designed so that the workflow can be repeated from a clean Python environment and its output files inspected.

Before submission

- ■ Install from requirements in a clean environment.
- ■ Run data validation, EDA, model training, tests, and linting successfully.
- ■ Inspect MLflow runs and save genuine screenshots.
- ■ Build and run Docker; verify `/health`, `/predict`, and `/metrics`.
- ■ Deploy to Minikube or cloud Kubernetes and save genuine evidence.
- ■ Record the requested short end-to-end video.
- ■ Push the project to GitHub and add the real repository URL.
- ■ Update this report with actual run results and any issues encountered.

References

1. Janosi, A., Steinbrunn, W., Pfisterer, M., & Detrano, R. (1989). *Heart Disease*. UCI Machine Learning Repository. https://doi.org/10.24432/C52P4X
2. Scikit-learn documentation. https://scikit-learn.org/
3. MLflow documentation. https://mlflow.org/docs/latest/
4. FastAPI documentation. https://fastapi.tiangolo.com/
5. Docker documentation. https://docs.docker.com/
6. Kubernetes documentation. https://kubernetes.io/docs/
7. Prometheus documentation. https://prometheus.io/docs/

Repository URL: **To be added after pushing to the student's GitHub account.**